

MILESTONE – 4 DEPLOYMENT

ExoHabitAI – Deployment Documentation

1. Objective

This document provides a detailed, step-by-step procedure for deploying the **ExoHabitAI** application so that it becomes publicly accessible through a web URL.

The deployment integrates:

- Trained Machine Learning Model
- Flask Backend API
- Frontend User Interface
- Production Server

The goal is to make the system accessible online for real-time habitability prediction.

2. Deployment Overview

The deployed application follows the architecture below:

User → Frontend → Flask API → ML Model → Response → Display Results

Flow Explanation

1. User enters planetary parameters in the frontend.
2. Frontend sends request to Flask API.
3. Flask loads trained ML model.
4. Model predicts habitability.
5. JSON response is returned.
6. Frontend displays results.

3. Deployment Platform Options

The application can be deployed using:

- **Recommended:** Render
- Alternative: Hugging Face
- Alternative: Streamlit

Render is recommended for production-ready Flask deployment.

4. Pre-Deployment Checklist

Before starting deployment, ensure:

Final ML model is saved inside models/ folder

Flask backend runs locally without errors

Frontend successfully connects to backend

requirements.txt is updated

.gitkeep files exist (if required)

Latest code is pushed to GitHub

Failure to complete these steps may cause deployment errors.

5. Backend Preparation

Step 1: Verify app.py

Ensure it includes:

- Flask app initialization
- Model loading using joblib
- API routes (/predict, /rank)
- JSON response handling

Step 2: Add Main Execution Block

Add the following at the bottom of app.py:

```
if __name__ == "__main__":  
    app.run(debug=False)
```

This ensures proper production execution.

6. Requirements File

Create or verify requirements.txt contains:

```
numpy  
pandas  
matplotlib  
seaborn  
scikit-learn  
xgboost  
flask  
joblib  
requests  
plotly
```

gunicorn
flask-cors

All dependencies must be listed to avoid “Module Not Found” errors.

7. Procfile (For Production Deployment)

Create a file named:

Procfile

Add the following line:

web: gunicorn backend.app:app

△ Adjust the path if your Flask app is structured differently.

This file tells Render how to start your application.

8. Deployment on Render (Recommended Method)

Step 1: Create Account

1. Go to: <https://render.com>
2. Sign up using GitHub

Step 2: Create New Web Service

1. Click **New** → **Web Service**
2. Connect your GitHub repository:
3. <https://github.com/GKSJ-Deepvision/B13-ExoHabitAI>

Step 3: Configure Service

Set the following:

Environment:

Python

Build Command:

pip install -r requirements.txt

Start Command:

gunicorn backend.app:app

Then click **Create Web Service**.

Render will automatically:

- Install dependencies
- Build application
- Start server
- Generate public URL

12. Final Deliverables (Deployment Phase)

Each intern must submit:

- ✓ Live deployment URL
- ✓ GitHub repository link
- ✓ Screenshot of working application
- ✓ Confirmation that API endpoints function correctly
- ✓ Deployment documentation

Conclusion

This document outlines the complete deployment procedure for ExoHabitAI using:

- Flask Backend
- Machine Learning Model
- Production Server (Gunicorn)
- Cloud Hosting via Render

Following these steps ensures successful public deployment of the application.

Github Repository Link :

<https://github.com/GKSJ19/ExoHabitAI/tree/Kala-Venkata-Naga-Sri-Yasaswini>

Public Render.com url :

<https://exohabitai-kvnsri-yasaswini.onrender.com>